

Module3 Distributed and Hybrid Programming

TaskM3.T1P: MPI, OpenMP and OpenCL Matrix Multiplication

Overview of the task

To fulfill the requirements of this task, you will need to demonstrate your skills to use MPI and OpenCL in C/C++ to speed up sequential program. In this task, we use matrix multiplication as the example problem. See here (https://en.wikipedia.org/wiki/Matrix_multiplication)

Submission Details

Please make sure to provide the following:

- Source code of the **MPI** matrix multiplication program,
- Source code of the **MPI** and **OpenMP** (hybrid **MPI** to nodes and **OpenMP** in the nodes) program,
- Source code of the **MPI** and **OpenCL** (hybrid **MPI** to nodes and **OpenCL** in the nodes) program. Alternatively, if **OpenCL** doesn't work properly within your **MPI** cluster, run **OpenCL** on a single computer and reflect on the performance gains/losses in comparison to other solutions in this task.
- Evaluation of your program on different input sizes and number of threads in each of these three programs.

Instructions

1. Modify your matrix multiplication program to use **MPI** to distribute work on nodes.
2. Evaluate the performance of your program vs sequential and multi-threaded solution.
3. Modify your code to use **OpenMP** on the nodes.
4. Evaluate the performance of your program vs the other two.
5. Modify your code to use **OpenCL**.
6. Evaluate the performance of your programs.
7. Document your results and present your findings
8. Submit your task as detailed on the submission details section above to OnTrack.

Assurance of Learning. Report and Demonstration Requirements

- Teaching staff may ask extra questions in chat and/or in a short interview to confirm understanding and authorship of **MPI** and hybrid work.
- The student must run at least one stress test for the project workload that either: (a) does not compile, or (b) is **DNF** (did not finish), or (c) runs for too long (state how long was waited, for example "waited 10 minutes");

the student must state the machine specifications used (CPU model, number of cores/threads, RAM, OS) and, if known, the reason for **DNF** (for example, compile error, out of memory, timeout, or user

interrupted due to long wait).

- The student must show compilation commands (**MPI** build, OpenMP flags if used) and must show screenshots of the program running.
- The student must test with large matrix sizes and must show timing evidence for multiple N values (for example, 100, 500, 1000, or sizes suited to the machine). The student must choose input sizes based on their own computer specifications so that runtimes are not too close to compare and the results support clear conclusions.
- The student must vary the number of **MPI** processes and for hybrid runs must also vary threads per process and must report results clearly.
- Every timing shown must include N, process count, thread count and the exact run command used (for example, the **mpirun** line).
- The student must show a correctness check and must explain any poor scaling or slowdown using clear reasons (start cost, data transfer cost and shared resource limits).
- The student must explain the **MPI** data split and the communication method used and must relate results to message cost and balance of work.
- The student must compare the **OpenCL** result against at least one earlier CPU baseline from previous tasks (for example, **sequential**, **pthread**s, **OpenMP**, or **MPI**) and must discuss the outcome; if the **OpenCL** run is CPU-only, the student must state that it is CPU-only and discuss how it compares to those CPU baselines.